Soft Comput
DOI 10.1007/s00500-016-2154-6

METHODOLOGIES AND APPLICATION

An intelligent power consumption model for virtual machines under CPU-intensive workload in cloud environment

Wentai Wu[1] · **Weiwei Lin**[1] · **Zhiping Peng**[2]

© Springer-Verlag Berlin Heidelberg 2016

Abstract Cloud computing has gained enormous popularity by providing high availability and scalability as well as on-demand services. However, with the continuous rise of energy consumption cost, the virtualized environment of cloud data centers poses a challenge to today's power monitoring system. Software-based power monitoring is gaining prevalence since power models can work precisely by exploiting soft computing methodologies like genetic programming and swarm intelligence for model optimization. However, traditional power models barely consider virtualization and have drawbacks like high error rate, low feasibility as well as insufficient scalability. In this paper, we first analyze the power signatures of virtual machines in different configurations through experiments. Then we propose a virtual machine (VM) power model, named CAM, which is able to adapt to the reconfiguration of VMs and provide accurate power estimating under CPU-intensive workload. We also propose two training methodologies corresponding to two typical situations for model training. CAM can estimate the power of a single VM as well as a physical server hosting several heterogeneous VMs. We exploited public Linux benchmarks to evaluate CAM .The experimental results show that CAM produced very small errors in power estimating for both VMs (4.26 % on average) and the host server (0.88 % on average).

Keywords Cloud computing · Power consumption · Virtual machine · vCPU · Power model

Communicated by V. Loia.

W. Wu and W. Lin contributed equally to this work.

✉ Weiwei Lin
linww@scut.edu.cn

Wentai Wu
cswuwt@mail.scut.edu.cn

[1] School of Computer Science and Engineering, South China University of Technology, Guangzhou 510006, China

[2] College of Computer and Electronic Information, Guangdong University of Petrochemical Technology, Maoming, China

1 Introduction

With the unprecedented development in information technology, the demand for highly elastic and scalable computing capability is growing rapidly. As a new distributed-computing paradigm, cloud computing takes advantage of multiple levels of virtualization. For example, Infrastructure-as-a-Service (IaaS) exploits hardware virtualization to maximize the utilizations of computing and storage devices by pooling these resources. However, as the scale of cloud data centers keeps growing rapidly, the huge energy consumption cost has become a major burden for cloud service providers. Report showed that the electricity cost of ICT devices occupies 42 % of their monthly operation budgets (Hamilton 2009). Scalability, reliability and data security (Li et al. 2015a) were major concerns for cloud computing, but now high energy consumption has become another challenging issue. To address it, much more attention should first be paid to the power monitoring in cloud data centers.

In order to solve the prominent problem of high energy cost, it is necessary to cast our sights on how to deploy more feasible power monitoring in cloud environment. Power monitoring is a basic module for data center energy management because accurate and real-time power data are the key information for energy-aware scheduling in virtualized clusters. On the one hand, as the government restricted electricity provision, cloud service providers (CSP) have

Published online: 25 April 2016

to monitor system power in a fine-grained style to perform strict power capping. On the other hand, energy saving keeps CSPs from suffering high energy cost and thus contributes a lot to increasing their profit. On this point, real-time power data are the most important reference for balancing regional workloads and conducting dynamic task scheduling. For instance, Amazon manages its distributed clusters by applying energy-aware strategy which is adaptive to regional power consumption and electrovalence.

Traditional power monitoring relies on physical devices such as external power meters and embedded measuring modules. However, cloud data centers are not only large-scaled and highly distributed, but also expand frequently leading to server heterogeneity. Measuring servers' power by physical devices has many drawbacks such as coarse-grained power data, difficult deployment, high cost and bad scalability. More importantly, cloud servers are generally virtualized. So if CSP wants to gather fine-grained power data at the virtual machine (VM) level, it is impossible to satisfy the requirement by only installing physical meters or circuit-based power modules. As an alternative, in such a virtualized environment, estimating system power through software method has been widely recognized. Software-based power monitoring has major advantages including high feasibility and low deployment cost. Meanwhile, it is still able to warrant accurate load prediction and power estimation by means of soft computing methodologies mentioned in Messias et al. (2015), Aliev et al. (2002) and Arroba et al. (2014).

For software-based power monitoring, the utilizations of resources must be sampled on a regular basis to quantify system's workload. And essentially the power model embedded in the monitoring software exploits the resource utilizations to realize real-time power estimating. Thus, much attention has been paid to the research on power models. On the one hand, power model serves as the essential part in software-based power metering. On the other hand, power models are often used to evaluate the effectiveness of resource scheduling algorithms (e.g. Peng et al. 2015; Lin et al. 2015b) because of its feasibility. However, currently available power models proposed by previous researches have defects listed below:

- Only being feasible for physical machines (PMs) and components (Luo et al. 2014; Arroba et al. 2014). Many traditional power models were built based on physical servers or specified components. In other words, these models cannot adapt well to virtualized environment.
- Dilemma of determining the expression of power model. Specifically, linear model is simple in form and easy to train, but it produces rather large error. More complex power models like polynomial are proved to be accurate, whereas they may suffer difficulties in training as well as the problem of over-fitting.
- Bad scalability. For most VM power models, a set of parameters is only suitable for one certain configuration of VM. This leads to repeated training when the target VM is reconfigured.
- Neglecting the difference between a VM and a PM in respect to the power signature. Our findings show that the VM and its host server have different power behaviors. In other words, high error may be produced if we estimate a VM's power by treating it as a normal process on the host.

In order to find a better solution for estimating the power of VMs, we were inspired by Chen et al. (2012) to build a workload-oriented VM power model. We carried out experiments to observe VM power signature. By installing an external power meter and running one VM every time, we sampled the power data of virtual machines in different configurations. Then by comparing the data with the power of a non-VM process on the same host, we conclude that there is an obvious difference between a VM and a non-VM process (i.e. the physical server) in power behavior. Besides, we looked into the power curves of VMs equipped with different numbers of v-cores and found that they are similar. Based on this fact, we in this paper propose a Core-Aware virtual machine power Model (CAM), which estimates VM power by monitoring vCPU utilization. CAM is adaptive to the dynamic reconfiguration of vCPU after one-time training. Meanwhile, we designed two different methodologies for the model training in different situations in data centers. Finally, we also evaluated the accuracy of CAM in experiments. The results illustrate that CAM provides higher accuracy in estimating single VM's power than linear model and cumulative model. The average relative error of CAM is 4.26 %. Besides, we created multiple heterogeneous VMs on a server and then made them running different CPU-intensive applications. In this case, we proved that CAM is also able to estimate the total power of physical server in a real-time manner with low errors (0.88 % averagely).

This paper is organized as follows: Related work about power modeling is introduced briefly in Sect. 2. Section 3 is about the observation and analysis of VM power signature. Then we propose our core-aware virtual machine power model and the corresponding training methodologies in Sect. 4. Model evaluation is given in Sect. 5 and we make conclusion in Sect. 6.

2 Related work

The core of software-based power monitoring is power model (Ye et al. 2012). Generally, power models can be divided

into two categories: models based on system resource utilization and models based on performance counter. For the first type, system resource utilization refers to the usage of hardware components (e.g. processor) or virtualized resources (e.g. vCPU). Its main idea is to find an appropriate function, which treats resource utilization as input and convert it to estimated power as output. Basmadjian et al. (2011) attempted to estimate power consumption of ICT devices by modeling several physical components including CPU, RAM, disk, etc. The model proposed in the paper includes nearly all physical components that make major contribution to system power, but it overlooks virtualization and application types. Power models based on performance counter need to sample and count performance events. For example, a possible approach to estimate RAM power is to count LLC (Last level Cache) misses. Colmant et al. (2015) proposed to sample the count of unhalted-cycles for each application on each physical core. In essence, this process-level power model treats a VM as a process. However, it takes no consideration the impact brought by virtualization on the actual power consumption in each cycle. There are two main problems for performance counter-based models. The first one is that system events are diverse and not easy to sample. Second, this kind of model is not that explainable. Thus, in this paper we select to model VM power consumption based on resource utilization.

Accurate power estimating relies on a proper power model. Linear power model is widely used in researches including Ye et al. (2012), Basmadjian et al. (2011), Kansal et al. (2010) and Lee and Zomaya (2012). In addition to that, linear model is also popular in evaluating resource scheduling algorithms (Sampaio and Barbosa 2014; Beloglazov et al. 2012) and virtual machine placement strategies (Lin et al. 2015a). However, Hsu and Poole (2011) surveyed the power data of 177 different types of servers provided by SPECpower_ssj2008 between 2007 and 2010. They concluded that servers' power signatures have changed, which leads to higher error rate if we continue to estimate servers' power using linear model. To solve this problem, many other power models in different forms are proposed [e.g. polynomial model Luo et al. 2014; Arroba et al. 2014 and sublinear model Hsu and Poole 2011. Theoretically, polynomial model is precise, whereas it may be complicated in training parameters. Besides, including too many parameters may also lead to over-fitting. Some scholars try to introduce more factors to make power model more accurate, but the experiment results from Arroba et al. 2014 show that only three parameters are valid after they trained their power model via Particle Swarm Optimization (PSO). As for multi-core processors, some researches (e.g. Basmadjian et al. 2011; Luo et al. 2013; Kim et al. 2011)] estimated CPU power exploiting the cumulative model. Based on the assumption that cores are independent of each other, this model calculates the power of each core separately. However, cumulative model produces higher error when the number of equipped cores increases.

Some sophisticated methods and machine-learning algorithms can be applied to train power models. Luo et al. (2014) proposed to use SVR (Support Vector machine Regression) and polynomial with lasso regression in model training. But practically, for concise models based on linear function or power function, it is still effective to train them through statistical methods like linear regression as mentioned in Song et al. (2013). Virtualization is common in modern data centers; some previous researches (e.g. Luo et al. 2014; Song et al. 2013; Tian et al. 2014) only focused on physical machines and cannot provide VM-level power estimating. Power models are frequently used in evaluating VM allocation/migration algorithms like Li et al. (2011) as well as virtual resource scheduling (Beloglazov et al. 2012; Zhao et al. 2014; Lin et al. 2014). To design a power estimate module, programmers prefer to use concise and feasible power models with fewer parameters. This helps to simplify the evaluation especially in simulated cloud environment like CloudSim (Calheiros et al. 2011).

Some researchers tried to build instruction-level power models (Leite et al. 2014; Capra et al. 2012). The biggest problem of this method lies in the difficulty of measuring the power of every machine instruction through experiments. A sample of this kind of model was proposed by Leite et al. (2014), but it suffers an error rate of up to 18.18 %.

3 VM power signature

For a physical machine, power signature (Laughman et al. 2003) shows the relationship between its power consumption and the workload level. Similarly, VM power signature represents how VM's power changes under varying workloads. Typically, system workload can be quantified as resource utilization. In other words, we have $P_{vm} = f(u)$ where P_{vm} is the current power of VM and u represents the usage of a certain resource. In modern computer system, CPU has been proved to be the major power consumption component (Basmadjian et al. 2011), especially in the worknodes mainly running CPU-intensive applications. Most of the relevant researches adopted power models based on CPU utilization.

However, we were facing two challenges if we want to find appropriate model that fits VM signature well. The first one is about how to determine the function f and the other one is how to quantify VM's workload by an effective metric that can be acquired easily.

In previous researches such as Basmadjian et al. (2011), Kansal et al. (2010) and Sampaio and Barbosa (2014), linear model was widely used since it is simple and able to provide relatively low-error estimation. However, with the

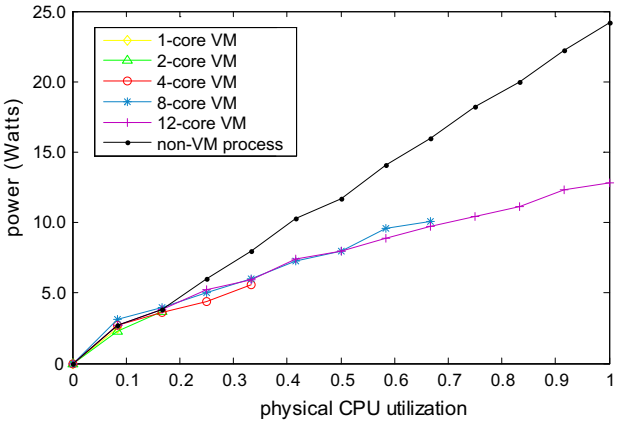


Fig. 1 The power signatures of VMs and a non-VM process

constant improvement on processor architecture, the power signatures of physical servers have changed (Hsu and Poole 2011). Continuing to use linear model may produce higher error. The power signature of a virtual machine is closely related to the physical machine (PM) that hosts it, but still there is a big difference. On the one hand, all servers have idle power while this value for a VM should be defined to zero. On the other hand, virtualization has a certain influence on VM's power. Thus traditional power models for PMs may not be suitable for VMs. Besides, some researches proposed to treat a VM as a process (Colmant et al. 2015), which makes it convenient to monitor the amount of resource that a VM occupies at hardware level. However, though a running VM is a kind of process indeed, the VM process differs a lot from a non-VM process with respect to power signature due to virtualization.

In order to find a feasible VM power model, we first conducted experiment on a physical server to observe the power signatures of VMs and a non-VM process.

The server model is Dell PowerEdge R730 equipped with dual 6-core Intel Xeon E5-2603 processors. CPU performance was set to *Max Performance* in BIOS. From the result (Fig. 1), it is easy to find out the obvious difference between the power consumption of the VMs and a normal process. To be more detailed, a VM probably consumes less power than a non-VM process when occupying equal amount of CPU resources. This is because a number of time slices are used to process instructions in the virtualization layer, so the actual CPU time consumed by the VM itself is reduced. Detailed explanation is beyond the scope of this paper. By analyzing the VM power curves in Fig. 1, we also point out that the power of VM increases in a sub-linear manner, which was substantiated by Hsu and Poole (2011). Last but not the least, we see that VMs in different configurations have the similarity in power signature, whereas there is a large difference between a VM and a non-VM process.

4 Core-aware virtual machine power model (CAM)

It is necessary to build a power model for virtual machines considering VM power signatures. According to the observation of our experiment showed in Fig. 1, we first use a power function of CPU utilization to express VM power:

$$p_{vm}(u_p) = \alpha \cdot u_p^{\beta}, \tag{1}$$

where u_p refers to CPU utilization, and αandβ are system parameters that satisfy $\alpha > 0, \beta > 0$. When β is in (0, 1], function p_{vm} is concave. Otherwise, it is convex. Empirically, concave function is suitable for describing server's power signature (Hsu and Poole 2011) and we found that it is similar for VMs. The main reason is that the energy consumption of a VM is mainly produced on the processor (Tesfatsion et al. 2014), especially under CPU-intensive workload. It is notable that u_p in (1) indicates the utilization of the physical CPU, so this model works with the premise that only one VM is running on the host without any other workload.

To make up this downside, we in this paper regard VM as an independent worknode so that it is possible to gain fine-grained power data without requiring CPU utilization from the host. The CPU resource allocated to a VM can be pre-configured. The cores allocated to a VM can be regarded as a virtual unit, namely vCPU. It is convenient to obtain vCPU utilization via API provided by guest-OS.

Exploiting the relationship between CPU and vCPU utilization (Guérout et al. 2013), we use vCPU utilization u_v to express u_p via this formula: $u_p = \frac{n}{N} \cdot u_v$. Then we have

$$p_{vm}(u_v, n) = \alpha \cdot \left(\frac{n}{N}\right)^{\beta} \cdot u_v^{\beta}, \tag{2}$$

where u_v is vCPU utilization that satisfies $u_v \in [0, 1]$, n represents the number of cores in the vCPU and N is the total number of cores in the physical CPU. The value of α and β depends on the hardware environment. Thus, to calculate the optimal parameters in current system, a certain amount of data are required to train the model.

It has been mentioned that the VM power signature depends on the host hardware configuration (Kansal et al. 2010). Considering the different conditions of servers in data centers, we further propose two different training methodologies. The first one is for the training on servers not deployed yet (Sect. 4.1) and the second one for servers already kept in service (Sect. 4.2). Hereinafter, we refer these two training situations as *situation A* and *situation B*, respectively.

Situation A is the typical training environment for both PM and VM power models. In this case, the server is completely free of any workload before we start to sample power data. Thus only one VM is powered on and only the target benchmark(s) will be run to produce VM workload. The

power of a single VM can be easily acquired in situation A where CAM can be trained simply via linear regression. However, in situation B, multiple virtual machines are hosted by the target server and it is not allowed to interrupt the service. In this case, only the power consumption of the whole server is available. Hence we should take the workloads of all the co-located VMs into consideration and use an iterative algorithm to optimize CAM's parameters.

4.1 Model training: situation A

At the beginning of the deployment or before a brand new type of server comes into service, if the CSP already plans to adopt software-based power monitoring, then it is a easy work to train CAM model. Model training in this situation requires power data sampled by running simple or mixed testing programs on the VM. More specifically, supposed that host-OS and virtualization software have been set up, the administrator needs to power on a virtual machine, run the benchmark(s) and record vCPU utilization u_v as well as the power p displayed on the physical meter connected to the server. It is suggested to reconfigure the vCPU setting and repeat the test. More VM power data of different configurations are helpful to optimize the parameters of CAM.

Let p_{idle} denote server idle power. Since we kept only one VM running every time, it could be regarded that the dynamic part of server power all comes from the running VM:

$$p_{vm} = p - p_{idle} \tag{3}$$

Let $i = 0, 1, 2, \ldots$ denote the sequence numbers of data samples. VM power $p_{vm,i}$ can be calculated from the measured power p_i via (3). In order to obtain optimized α and β, it is necessary to transform expression (2), which is nonlinear. We made use of the power function's characteristic and transformed the expression into a form suitable for linear regression. Take the logarithm of the both sides of (2); we have

$$\begin{aligned} \ln p_{vm} &= \ln\left(\alpha \cdot \left(\tfrac{n}{N}\right)^{\beta} \cdot u_v^{\beta}\right) \\ &= \ln\alpha + \beta \ln\left(\tfrac{n}{N} u_v\right) \end{aligned} \tag{4}$$

Before training, we preprocessed the raw data by replacing all zeros using a small value to make their logarithms calculable. To be more specific, we used (0.01, 0.01) to replace (0, 0). By making $Y = \ln p_{vm}$, $X = \ln(\frac{n}{N} u_v)$ and $\alpha' = \ln\alpha$, (4) can now be expressed as below:

$$Y = \alpha' + \beta \cdot X \tag{5}$$

With the training data set $D = \{(n_i, u_{v,i}, p_{vm,i}) | i \in [1, I]\}$, the optimal values of parameters α and β can be obtained by means of linear regression with ordinary least squares estimation. We used this methodology to train the proposed core-aware VM power model (CAM) under the experimental setup illustrated in Sect. 5.

4.2 Model training: situation B

Now consider another situation: the administrator decides to collect the power information of all VMs on all servers. However, CAM is never trained on some types of servers. Meanwhile, to guarantee the QoS (Quality of Service) of IaaS, now it is not permitted to interrupt the workload on all these cloud servers. Assume that there are m VMs running the target server; the numbers of vCPU cores of the VMs are $n_1, n_2, \ldots, n_m$, respectively, and currently their vCPU utilization are $u_{v,1}, u_{v,2}, \ldots, u_{v,m}$, respectively. Let N be the number of cores equipped in this server and $p_{dynamic}$ the dynamic power currently. So we have:

$$p_{dynamic} = p - p_{idle} \tag{6}$$

$$p_{dynamic} = \sum_{k=1}^{m} p_{vm,k} \tag{7}$$

We can estimate the power of each VM via CAM and the estimated system power equals

$$\tilde{p}_{dynamic} = \sum_{k=1}^{m} \tilde{p}_{vm,k} = \sum_{k=1}^{m} \left[\alpha \cdot \left(\frac{n_k}{N}\right)^{\beta} \cdot u_{v,k}^{\beta}\right] \tag{8}$$

We assume none of the VMs on the target host are shut down or migrated and no new VM is powered on in a certain period. The administrator samples the data I times including the server's power p_i and vCPU utilization $u_{v,k,i}$ of each VM at moment i, where $i = 1, 2, \ldots, I$ denotes the sequence numbers of data and $k = 1, 2, \ldots, m$ denotes VM numbers. Then for each point in the data set $D = \{(u_{v,1,i}, u_{v,2,i}, \ldots, u_{v,m,i}, p_i) | i \in N^*, i \in [1, I]\}$, the error of CAM can be calculated via the expression $\varepsilon_i = \tilde{p}_i - p_i$. $\tilde{p}_i$ denotes the estimated server power and it can be calculated via $\tilde{p}_i = p_{idle} + \tilde{p}_{dynamic}$. Thus we have:

$$\begin{aligned} \varepsilon_i &= (p_{idle} + \tilde{p}_{dynamic}) - p_i \\ &= p_{idle} + \sum_{k=1}^{m} \left[\alpha \cdot \left(\tfrac{n_k}{N}\right)^{\beta} \cdot u_{v,k}^{\beta}\right] - p_i \end{aligned} \tag{9}$$

In this paper, we exploit Least Squares Estimation to construct the target function $\delta(\alpha, \beta)$ as follows:

$$\delta(\alpha, \beta) = \sum_{i=1}^{I} \varepsilon_i^2 \tag{10}$$

Then solving the optimized values of α and β is equivalent to the unconstrained optimization expressed in (11):

$$\min \delta(\alpha, \beta) \tag{11}$$

Theoretically, optimal α and β can be obtained by solving the equations below:

$$\begin{cases} \dfrac{\partial \delta(\alpha, \beta)}{\partial \alpha} = 0 \\ \dfrac{\partial \delta(\alpha, \beta)}{\partial \beta} = 0 \end{cases} \tag{12}$$

However, it is generally not easy to get the explicit solution by combining (9), (10), (12). In practice, it is more feasible to exploit iterative algorithms to obtain the approximate solution. For instance, we can use ADM (Alternating Directions Method) to approach optimized model parameters by setting the initial values: $\alpha^{(0)} = 10.0$, $\beta^{(0)} = 0.5$, $\varepsilon_1 = 0.1$ and $\varepsilon_2 = 0.01$ empirically. The pseudo-code showed in Algorithm 1 describes the detailed training process using ADM:

Algorithm 1: CAM training in situation B

$\alpha = \alpha^{(0)}, \beta = \beta^{(0)}$
$\lambda_1^{(0)} = MAX_FLOAT$
$\lambda_2^{(0)} = MAX_FLOAT$
$k = 0$
while $\lambda_1^{(k)} > \varepsilon_1$ *or* $\lambda_2^{(k)} > \varepsilon_2$ **do**
  $k = k + 1$
  use linear search to find $\lambda_1^{(k)}$ that minimizes $\delta(\alpha^{(k-1)} + \lambda_1^{(k)}, \beta^{(k-1)})$
  $\alpha^{(k)} = \alpha^{(k-1)} + \lambda_1^{(k)}$
  use linear search to find $\lambda_2^{(k)}$ that minimizes $\delta(\alpha^{(k)}, \beta^{(k-1)} + \lambda_2^{(k)})$
  $\beta^{(k)} = \beta^{(k-1)} + \lambda_2^{(k)}$
end
$\alpha = \alpha^{(k)}, \beta = \beta^{(k)}$

4.3 Advantages of CAM

By analyzing VM power signature and introducing a new variable, we propose a new power model that works at the VM-level. CAM can adapt to the dynamic reconfiguration of vCPU and thus it is core-aware. In the meantime, we discuss different situations in which the model parameters are optimized by different methods. For a server not deployed yet (situation A), it is convenient to collect the power data of a single VM and obtained optimized α and β via simple linear regression. It is strongly suggested to sample bigger data set. In situation B, we propose a training methodology by which CAM can be trained on a server without interrupting any VM running user applications currently. It exploits an iterative algorithm but does not require huge amount of data (i.e. big effort in sampling). We conclude three advantages of CAM:

- Good adaption to vCPU reconfiguration. By introducing n to represent the number of cores in vCPU, the proposed model CAM is able to estimate the power of VMs in different configurations without training multiple sets of parameters. This feature provides great convenience for power monitoring of heterogeneous virtual clusters.
- High feasibility and low-cost training in practical situations. CAM has only two system parameters and uses a single signal (vCPU utilization) for VM power estimating. Convenient training methods are provided especially on servers not deployed yet because in this case the model can be trained by performing linearization and linear regression.
- VM-level power metering. CAM provides accurate and real-time power estimating. It is also notable that CAM is not only capable of estimating VM power, but also it can estimate power consumption of applications running in the VM.

5 Model evaluation

To evaluate the accuracy of CAM after training, we carried out experiments divided into two parts. In the first part (Sect. 5.2) we evaluated the error of CAM on estimating power consumption of a single VM. We differentiated vCPU configurations every time and sampled vCPU utilization as well as power data. In the second part of our experiment, we started up three heterogeneous virtual machines on a physical server. The VMs were running CPU-intensive workload (different applications). Physical meter with data recording function was installed to sample this server's power and a Linux tool for resource monitoring was kept running for a period to record each VMs' vCPU utilization, respectively.

5.1 Experimental setup

We conducted the experiment by installing virtual machine manager (VMM) on a Dell PowerEdge R730 with two Intel Xeon E5-2603 six-core processors. The frequency of each core is 1.60GHz. The server is also equipped with a 48-GB RAM as well as a 1-TB HDD. The operating system is Windows Server 2008 R2 and CPU performance is set to *Max Performance*. VMM version is Oracle VirtualBox 4.2.16 and we installed Ubuntu 12.04 LTS as Guest-OS in all virtual machines. We used Wattsup?PRO[1] with a USB interface connected to another PC to measure server power. This device can measure power with an error smaller than 0.5 %. We set

[1] Watts up? Plug load meters. https://www.wattsupmeters.com/.

its sampling interval to 1s and the data are stored in a log on the PC. In order to collect sufficient utilization data of vCPU for model training, we made use of CPU-intensive benchmarks to generate varying VM workload by controlling the number of running threads.

We in this paper exploit the methodology mentioned in Sect. 4.1 to train CAM. Power and vCPU usage were sampled in the process of running a single VM on the target Dell server. We used linear regression to train CAM model. As a result, the VM power model after training is shown as below:

$$p_{vm}(u_v, n) = 13.114 \cdot \left(\frac{n}{N}\right)^{0.699} \cdot u_v^{0.699} \tag{13}$$

CAM can be applied to estimate not only the power of virtual machines, but also physical servers that host multiple VMs. We evaluated the accuracy of CAM on both a single VM and a physical server. It is worthwhile to mention that the term "error" used in following sections refers to residual. A residual is the difference between a measured value and the estimated value provided by a power model.

5.2 Single VM

We first evaluated the accuracy of CAM on estimating the power of a single VM. One virtual machine was powered on every time and the Linux benchmark SysBench[2] was used to generate workload. We varied vCPU workload by adjusting the number of running threads. Five VMs of different configurations were tested one by one and their numbers of equipped cores are 1, 2, 3, 4 and 5 respectively. 2GB memory was allocated to each VM and the I/O workload was negligible. The measured power and estimated power by CAM are compared in Fig. 2a and the confidence intervals of relative error are shown in Fig. 2b. Confidence level is set to 90 %. Besides, for comparison, we also calculated the error bounds of linear model (Kansal et al. 2010) (Fig. 3a) and cumulative model (Luo et al. 2013) (Fig. 3b) on the same data set. Together the 3 models are compared in Fig. 4.

From Fig. 2 we can see that the sub-linear CAM model profiles well the power signatures of VMs. In particular, by introducing variable n, CAM can adapt to the change of vCPU configuration. It provides high accuracy in estimating the power of the VMs configured in our experiment with identical α and β. CAM can be regarded as an extension of linear model but it better fits the power curves of VMs, especially at the steep raising when vCPU utilization is low. CAM produced about 5 % lower error than linear model averagely. Cumulative model considers each core as an isolated power consumption unit and thus the power of VM equals the summary of power produced by each core (Colmant et al. 2015). One drawback of cumulative model is that it neglects

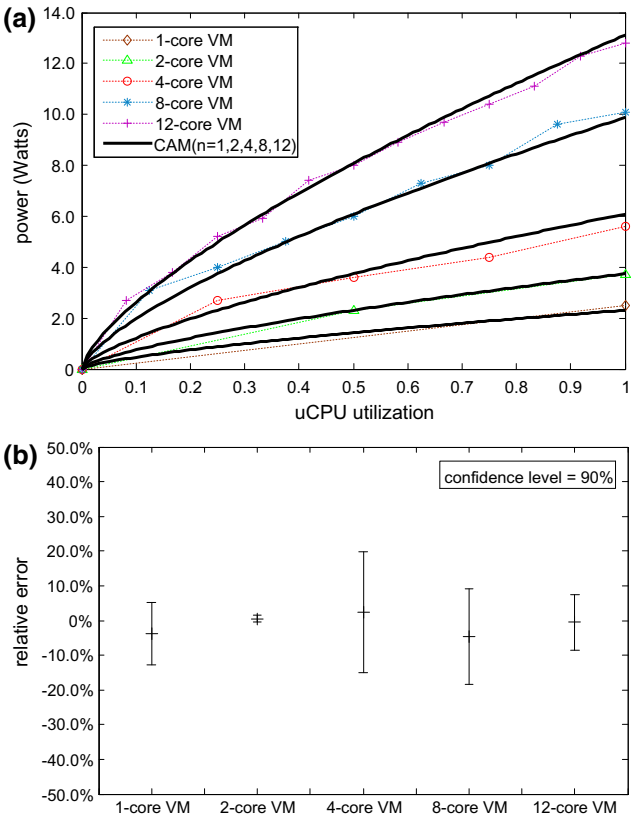


Fig. 2 **a** Estimated power from CAM and the observed power behaviors of VMs. **b** Relative errors of CAM

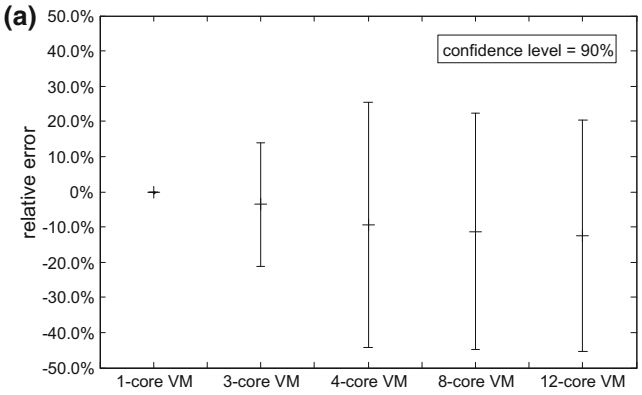
the interaction between multiple cores. From Fig. 3b it is easy to discover that cumulative model is likely to produce relatively large positive error (54.71 % on average). Given the confidence level, CAM probably produces smaller error than linear and cumulative models (Figs. 2, 3). As a conclusion, the comparison on mean relative errors shown in Fig. 4 illustrates that CAM provides the highest accuracy among the three models.

5.3 Server hosting multiple VMs

For cloud data center managers, it is necessary to gather the power conditions of servers periodically. For instance, every deployed server may host one to many virtual machines in IaaS, so fine-grained power monitoring for both VMs and servers is pretty helpful for effective energy-aware scheduling like performing VM migration. In this part of experiment, we started up multiple VMs and made them running different CPU-intensive applications. These VMs were co-hosted but heterogeneous. Again we exploited WattsUp? PRO meter to automatically record server power with sampling frequency set to 1Hz. Sysstat[3] worked as a Linux tool in each VM for recording the vCPU utilization every 1 second. Table 1

[2] SysBench: https://github.com/akopytov/sysbench.

[3] Sysstat: http://sebastien.godard.pagesperso-orange.fr/download.html.



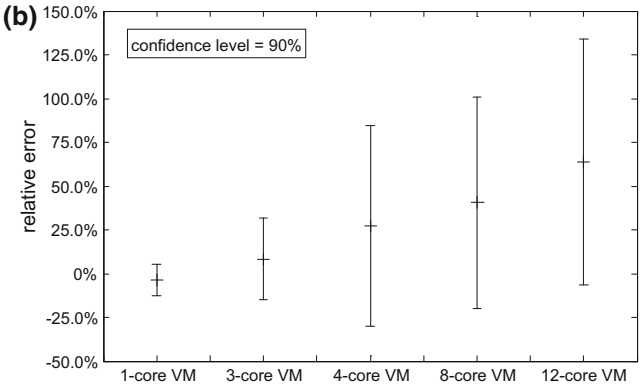


Fig. 3 **a** Relative errors of linear model. **b** Relative errors of cumulative model

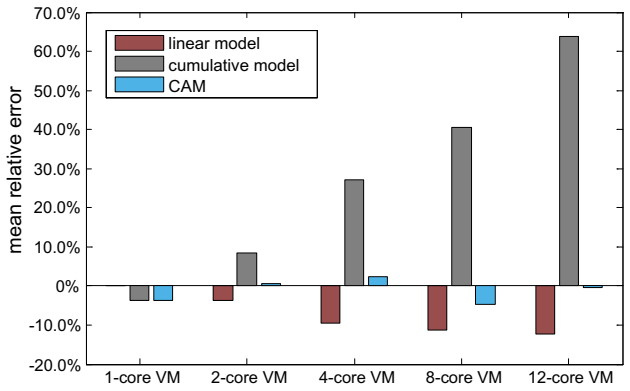


Fig. 4 This figure shows the accuracy of linear model, cumulative model and CAM on estimating VM power

shows the vCPU configuration and the running applications in each VM as well as their running time. The result and the statistics on estimation errors are shown in Figs. 5 and 6, respectively.

Totally, we measured the server's power by the power meter in a run of 70 s. The server was idle in the beginning 5 s and all the applications in the VMs stopped producing workload in the last 5 s. The estimated server power was calculated by adding up server idle power and every VM's power estimated from CAM (Fig. 5). The max and average absolute relative errors are also marked in Fig. 5.

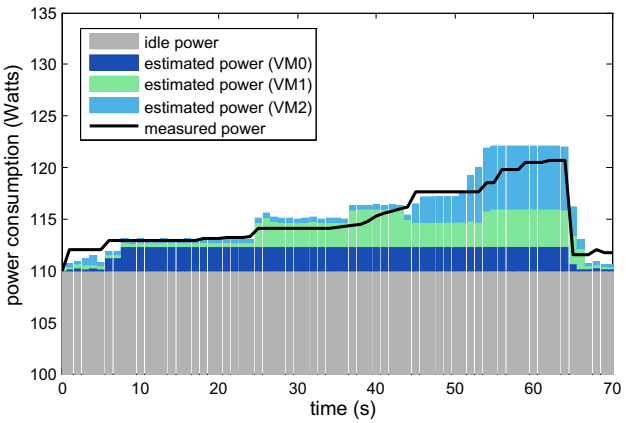


Fig. 5 Using CAM to estimate server power by aggregating the power of all running VMs in the 70-s run

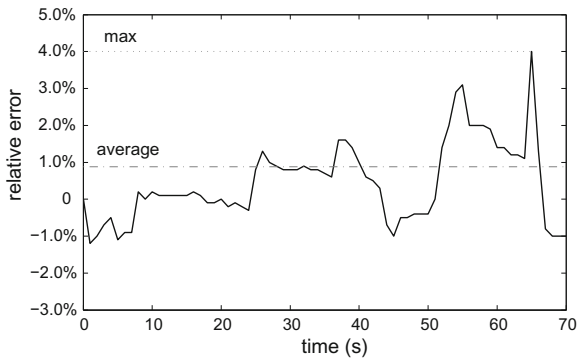


Fig. 6 The relative errors of estimated server power acquired by adding together three VMs power and server idle power

Table 1 Experimental setup of VMs, applications and the running time

Virtual machine	Config. of vCPU	Application	Running time (starts, ends)
VM_0	1-core	Nbench[4]	(5, 25)
VM_1	2-core	Fhourstone[5] + Dhrystone[6]	(25, 45)
VM_2	4-core	Ubench[7]	(45, 65)

[4] nbench: http://www.tux.org/~mayer/linux/bmark.html
[5] Fhourstone: http://tromp.github.io/c4/fhour.html
[6] Dhrystone: http://www.anime.net/~goemon/benchmarks.html
[7] Ubench: http://www.phystech.com/download/ubench.html

The black line represents measured power. We discovered that estimated power was more "sensitive" to the changing of vCPU utilization and thus rose rapidly when a new application started. That the observed power curve (bold black line) is steadier is mainly due to the smoothing on outputs by the physical meter's internal circuit. To conclude, CAM is also accurate in estimating power on physical servers which host multiple VMs. In this case, the max error of CAM is 4.04 % and the average error is below 1 %.

6 Conclusion

In recent years, cloud computing is providing high-avail ability, high-performance and elastic service for both individual and enterprise users. As the huge energy consumption turns out to be a prominent problem for cloud data centers, power monitoring purely via physical devices is no longer feasible for the virtualized environment. To address this issue, a VM-level power model, namely CAM, is proposed in this paper. Different from models based on physical machines, CAM can provide accurate and real-time power estimating for VMs. In particular, CAM is able to adapt to the reconfiguration of vCPU without training many times. We also proposed two training methodologies corresponding to different conditions of the target server. It is more convenient to train the model on a server not deployed yet. We exploited public Linux benchmarks to generate CPU-intensive workload in VMs and evaluated the accuracy of CAM. The results illustrate that CAM is precise in respect to estimating single VM's power and it only produces small errors on estimating the power of a server hosting multiple VMs.

Though a feasible virtual machine power model is proposed in this paper, we plan to extend our model for estimating the power of worknodes running I/O-intensive and data-oriented applications. We also plan to evaluate our power model under the workload of specified applications like data deduplication and encryption (Li et al. 2014a, b). Besides, we are going to develop corresponding power estimate modules and plant them into popular simulation-based cloud test beds like CloudSim. These modules are of great help to evaluate the performance of energy-saving scheduling strategies such as Beloglazov et al. (2012), Li et al. (2015b) and Wang et al. (2014).

Acknowledgments Thanks to the helpful comments and suggestions from the anonymous reviewers. This work is partially supported by the National Natural Science Foundation of China (Grant Nos. 61402183 and 61272382), Guangdong Natural Science Foundation (Grant No. S2012030006242), Guangdong Pro- vincial Scientific and Technological Projects (Grant Nos. 2014B 010117001, 2014A010103022, 2014A010103008, 2013B010202001 and 2013B090200021) and Fundamental Research Funds for the Central Universities, SCUT(No. 2015ZZ0098).

Compliance with ethical standards

Funding This study was funded by the National Natural Science Foundation of China (Grant Nos. 61402183 and 61272382), Guangdong Natural Science Foundation (Grant No. S2012030006242), Guangdong Pro vincial Scientific and Technological Projects (Grant Nos. 2014B 010117001, 2014A010103022, 2014A010103008, 2013B010202001 and 2013B090200021) and Fundamental Research Funds for the Central Universities, SCUT (No. 2015ZZ0098).

Conflict of interest Wentai Wu declares that he has no conflict of interest. Weiwei Lin declares that he has no conflict of interest. Zhiping Peng declares that he has no conflict of interest.

Ethical approval This article does not contain any studies with human participants or animals performed by any of the authors.

References

Aliev RA, Fazlollahi B, Vahidov R (2002) Genetic algorithms-based fuzzy regression analysis. Soft Comput 6(6):470–475. doi:10.1007/s00500-002-0163-0

Arroba P, Zapater M, Moya JM, Ayala JL, Olcoz K (2014) Server power modeling for run-time energy optimization of cloud computing facilities. Energy Procedia 62:401–410. doi:10.1016/j.egypro.2014.12.402

Basmadjian R, Ali N, Niedermeier F, Meer HD, Giuliani G (2011) A methodology to predict the power consumption of servers in data centres. In: Proceedings of the 2nd international conference on energy-efficient computing and networking. ACM press, New York, pp 1–10. doi:10.1145/2318716.2318718

Beloglazov A, Abawajy J, Buyya R (2012) Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing. Future Gener Comput Syst 28(5):755–768. doi:10.1016/j.future.2011.04.017

Calheiros RN, Ranjan R, Beloglazov A, Rose CA, Buyya R (2011) Cloudsim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms. Softw Pract Exp 41(1):23–50. doi:10.1002/spe.995

Capra E, Francalanci C, Slaughter SA (2012) Is software "green"? Application development environments and energy efficiency in open source applications. Inf Softw Technol 54(1):60–71. doi:10.1016/j.infsof.2011.07.005

Chen FF, Schneider JG, Yang Y, Grundy J, He Q (2012) An energy consumption model and analysis tool for cloud computing environment. In: Proceedings of the first international workshop on green and sustainable software. IEEE press, New York, pp 45–50. doi:10.1109/GREENS.2012.6224255

Colmant M, Kurpicz M, Felber P, Huertas L, Rouvoy R, Sobe A (2015) Process-level power estimation in VM-based systems. In: Proceedings of the tenth European conference on computer systems (EuroSys'15), Apr 2015, Bordeaux, France. ACM 14, pp 1–14. doi:10.1145/2741948.2741971

Guérout T, Montei Tl, Costa GD, Calheiros RN, Buyya R, Alexandru M (2013) Energy-aware simulation with DVFS. Simul Model Pract Theory 39:76–91. doi:10.1016/j.simpat.2013.04.007

Hamilton J (2009) Cooperative expendable micro-slice servers (CEMS): low cost, low power servers for internet-scale services. In: Proceedings of the conference on innovative data systems research (CIDR'09). http://dblp.uni-trier.de/db/conf/cidr/cidr2009.html

Hsu CH, Poole SW (2011) Power signature analysis of the SPECpower_ssj2008 benchmark. In: Proceedings of 2011 IEEE international symposium on performance analysis of systems and software (ISPASS). IEEE COMPUTER SOC press, Los Alamitos, pp 227–236. doi:10.1109/ISPASS.2011.5762739

Kansal A, Zhao F, Liu J, Kothari N, Bhattacharya A A (2010) Virtual machine power metering and provisioning. In: Proceedings of the 1st ACM symposium on cloud computing. ACM press, New York, pp 39–50. doi:10.1145/1807128.1807136

Kim N, Cho J, Seo E (2011) Energy-based accounting and scheduling of virtual machines in a cloud system. In: Proceedings of 2011 IEEE/ACM international conference on green computing and communications (GreenCom). IEEE press, New York, pp 176–181. doi:10.1109/GreenCom.37

Laughman C, Lee K, Cox R, Shaw S, Leeb S, Norford L, Armstrong P (2003) Power signature analysis. IEEE Power Energy Mag 1(2):56–63. doi:10.1109/MPAE.2003.1192027

Lee YC, Zomaya AY (2012) Energy efficient utilization of resources in cloud computing systems. J Supercomput 60(2):268–280. doi:10.1007/s11227-010-0421-3

Leite A, Tadonki C, Eisenbeis C, Melo AD (2014) A fine-grained approach for power consumption analysis and prediction. Procedia Comput Sci 29:2260–2271. doi:10.1016/j.procs.2014.05.211

Li Q, Hao QF, Xiao LM, Li ZJ (2011) Adaptive management and multi-objective optimization for virtual machine placement in cloud computing. Chin J Comput 34(12):2253–2264

Li J, Chen X, Li M, Li J, Lee PPC, Lou W (2014a) Secure deduplication with efficient and reliable convergent key management. IEEE Trans Parallel Distrib Syst 25(6):1615–1625. doi:10.1109/TPDS.2013.284

Li J, Huang X, Li J, Chen X, Xiang Y (2014b) Securely outsourcing attribute-based encryption with checkability. IEEE Trans Parallel Distrib Syst 25(8):2201–2210. doi:10.1109/TPDS.2013.271

Li J, Li JW, Chen XF, Jia CF, Lou WJ (2015a) Identity-based encryption with outsourced revocation in cloud computing. IEEE Trans Comput 64(2):425–437. doi:10.1109/TC.2013.208

Li J, Li Y, Chen XF, Lee P, Lou WJ (2015b) A hybrid cloud approach for secure authorized deduplication. IEEE Trans Parallel Distrib Syst 26(5):1206–1216. doi:10.1109/TPDS.2014.2318320

Lin WW, Tan L, Wang JZ (2014) Novel resource allocation algorithm for energy-efficient cloud computing in heterogeneous environment. Int J Grid High Perform Comput (IJGHPC) 6(1):63–76. doi:10.4018/ijghpc.2014010104

Lin WW, Xu SY, Li J, Xu LL, Peng ZP (2015a) Design and theoretical analysis of virtual machine placement algorithm based on peak workload characteristics. Soft Comput. doi:10.1007/s00500-015-1862-7

Lin WW, Zhu CY, Li J, Liu B, Lian H (2015b) Novel algorithms and equivalence optimisation for resource allocation in cloud computing. Int J Web Grid Serv 11(2):193–210. doi:10.1504/IJWGS.2015.068899

Luo L, Wu W, Tsai WT, Di D, Zhang F (2013) Simulation of power consumption of cloud data centers. Simul Model Pract Theory 39:152–171. doi:10.1016/j.simpat.2013.08.004

Luo L, Wu W, Zhang F (2014) Energy modeling based on cloud data center. J Softw 25(7):1371–1387. doi:10.13328/j.cnki.jos.004604

Messias VR, Estrella JC, Ehlers R, Santana MJ, Santana RC, Reiff-Marganiec S (2015) Combining time series prediction models using genetic algorithm to auto-scaling web applications hosted in the cloud infrastructure. Neural Comput Appl. doi:10.1007/s00521-015-2133-3

Peng ZP, Cui DL, Zuo JL, Li QR, Xu B, Lin WW (2015) Random task scheduling scheme based on reinforcement learning in cloud computing. Cluster Comput. doi:10.1007/s10586-015-0484-2

Sampaio AM, Barbosa JG (2014) Towards high-available and energy-efficient virtual computing environments in the cloud. Future Gener Comput Syst 40:30–43. doi:10.1016/j.future.2014.06.008

Song J, Li T, Wang Z, Zhu Z (2013) Study on energy-consumption regularities of cloud computing systems by a novel evaluation model. Computing 95(4):269–287. doi:10.1007/s00607-012-0218-8

Tesfatsion SK, Wadbro E, Tordsson J (2014) A combined frequency scaling and application elasticity approach for energy-efficient cloud computing. Sustain Comput Inf Syst 4(4):205–214. doi:10.1016/j.suscom.08.007

Tian Y, Lin C, Li K (2014) Managing performance and power consumption tradeoff for multiple heterogeneous servers in cloud computing. Cluster Comput 17(3):943–955. doi:10.1007/s10586-013-0326-z

Wang X, Wang Y, Cui Y (2014) An energy-aware bi-level optimization model for multi-jobs scheduling problems under cloud computing. Soft Comput. doi:10.1007/s00500-014-1506-3

Ye KJ, Wu ZH, Jiang XH, He QM (2012) Power management of virtualized cloud computing platform. Chin J Comput 35(6):1262–1285

Zhao H, Liu X, Li X (2014) Towards efficient and fair resource trading in community-based cloud computing. J Parallel Distrib Comput 74(11):3087–3097. doi:10.1016/j.jpdc.2014.07.005